GPT PROJECT - MILESTONE 3

Chau Tran, Ryan Nguyen, Max Bromberg, Muqi Xu

1 REQUIREMENT 4: FLASH ATTENTION

1.1 GRID AND BLOCK CONFIGURATION

In this implementation, we adopt the FlashAttention2 (Dao, 2024) algorithm. Our implementaion uses these parameters:

```
#define Q_TILE_SIZE    16        // Process 16 Q tokens per block
#define KV_TILE_SIZE   64        // Process 64 K/V tokens per iteration
#define WARP_SIZE      32        // 32 threads per warp
#define THREADS        128       // 128 threads per block (4 warps)
const int WMMA_M = 16;           // 16x16 output tile dimension
const int WMMA_N = 16;           // 16x16 intermediate tile dimension
const int WMMA_K = 8;            // TF32 matrix multiply shape (depth dimension)
```

- **Q_TILE_SIZE**: Number of query tokens processed per thread block
- **KV_TILE_SIZE**: Number of key/value tokens processed in each iteration
- **WARP_SIZE**: Standard NVIDIA GPU warp size (32 threads)

```
dim3 num_threads(THREADS);  // 128 threads (4 warps)
dim3 num_blocks(CEIL_DIV(T, Q_TILE_SIZE), NH, B);
float scale = 1.0 / sqrtf(HS);

// Dynamic shared memory: all float for TF32
// s_q(16*HS float) + s_K(64*HS float) + s_V(64*HS float) +
// s_qK(16*64 float) + s_o(16*HS float)
size_t smem_size = (Q_TILE_SIZE * HS + KV_TILE_SIZE * HS +
                    KV_TILE_SIZE * HS) * sizeof(float) +
                   (Q_TILE_SIZE * KV_TILE_SIZE + Q_TILE_SIZE * HS) * sizeof(float);

flash_attn_kernel_tf32<<<num_blocks, num_threads, smem_size>>>(
    q, k, v, attention_out, B, NH, T, HS, scale
);
```

In our current implementation, the grid is configured as: Grid $= \left(\left\lceil \frac{T}{Q_TILE_SIZE} \right\rceil, N_H, B\right)$, where The total number of blocks: $\lceil T/16 \rceil \times N_H \times B$. Where Each block is responsible for computing the attention output for: One Q tile (with 16 query tokens in this case) using index from $batchIdx.x$. One attention head (head index $batchIdx.y$). One batch (batch index $batchIdx.z$).

Each block has exactly 128 threads, which is 4 warps (since $128/32 = 4$ warps). Those 128 threads cooperatively load data into shared memory, find max values or group in to warp to independently computes one WMMA tile during GEMMs operations.

Dimension	Size	Meaning
X	$\lceil T/16 \rceil$	Number of Q tiles (tiles of 16 tokens)
Y	N_H	One per attention head
Z	B	One per batch

Table 1: Flash Attention Kernel Grid Dimensions

Algorithm 1 FlashAttention-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \ldots, L_{T_r}$ of size B_r each.
for $1 \le i \le T_r$ **do**
  Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
  On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
  for $1 \le j \le T_c$ **do**
    Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \mathrm{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \mathrm{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{O}_i^{(j)} = \mathrm{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
  end for
  On chip, compute $\mathbf{O}_i = \mathrm{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
  On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
  Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.
  Write L_i to HBM as the i-th block of L.
end for
Return the output $\mathbf{O}$ and the logsumexp L.

Figure 1: Flash Attention 2 (Dao, 2024) Pseudo Code

1.2 Flash Attention Implementation

1.2.1 Shared Memory Initization and Loading

Our implementation stores five main buffers in dynamic shared memory, all using `float` (FP32) for TF32 WMMA operations.

$$\begin{aligned} \text{smem_size} = (&2 \times Q_TILE_SIZE \times H_S && (1)\\ &+ 2 \times KV_TILE_SIZE \times H_S && (2)\\ &+ Q_TILE_SIZE \times KV_TILE_SIZE) \times \text{sizeof(float)}. && (3) \end{aligned}$$

- First, the query tile `s_q` holds $Q_TILE_SIZE \times H_S$ elements **(1)** because each block processes Q_TILE_SIZE query tokens, each of dimension H_S.
- The key and value tiles, `s_K` and `s_V`, each require $KV_TILE_SIZE \times H_S$ elements **(2)**. The kernel streams keys and values in tiles of KV_TILE_SIZE tokens per iteration, and every token again has H_S features.
- The intermediate score buffer `s_qK` has size $Q_TILE_SIZE \times KV_TILE_SIZE$ stored as `float`, storing all raw QK logits for one Q-tile against one KV-tile. This full matrix **(3)** is needed for softmax normalization and for later P@V computation.
- Finally, the output buffer `s_o` stores $Q_TILE_SIZE \times H_S$ **(1)** accumulated values in `float` before writing them back to global memory. Combining these terms yields the shared memory requirement.

Variable	Type	Size
s_q	float*	$Q_TILE_SIZE \times H_S$
s_K	float*	$KV_TILE_SIZE \times H_S$
s_V	float*	$KV_TILE_SIZE \times H_S$
s_qK	float*	$Q_TILE_SIZE \times KV_TILE_SIZE$
s_o	float*	$Q_TILE_SIZE \times H_S$

Table 2: Shared Memory Layout for TF32 WMMA Flash Attention Kernel

In addition to the dynamic region, the kernel allocates small static shared-memory arrays used for per-row softmax statistics, including the running maximum, previous maximum, and the running sum of exponentials for each of the 16 rows in the Q-tile.

```
__shared__ float s_block_max[Q_TILE_SIZE];     // Max per row in current tile
__shared__ float s_sum_exp[Q_TILE_SIZE];       // Sum of exp per row
__shared__ float s_prev_max[Q_TILE_SIZE];      // Previous max for rescaling
__shared__ float s_row_max[Q_TILE_SIZE];       // Running row maximum
```

Now that we have the layout of all needed memory, we need to load them on chip to compute. In our current implementation, all threads work together to process the shared memory. Thread iteration patterns use stride to distribute work:

```
// Initialize accumulator and statistics
for (int d = tid; d < Q_TILE_SIZE * D; d += blockDim.x) {
    s_o[d] = 0.0f;  // Output accumulator (fp32)
}
for (int t = tid; t < Q_TILE_SIZE; t += blockDim.x) {
    s_block_max[t] = -INFINITY;
    s_row_max[t] = -INFINITY;  // m_i (running maximum)
    s_sum_exp[t] = 0.0f;       // d_i (softmax denominator)
}
__syncthreads();

// Load Q tile once
for (int d = tid; d < Q_TILE_SIZE * D; d += blockDim.x) {
    int q_idx = d / D;
    int d_idx = d % D;
    int q_offset = q_tile_offset + q_idx;
    if (q_offset < T) {
        // Load from global memory, keep as float for TF32 WMMA
        s_q[d] = Q[batch_head_offset + q_offset * D + d_idx];
    } else {
        s_q[d] = 0.0f;
    }
}
// Initialize output buffer (float32)
for (int d = tid; d < Q_TILE_SIZE * D; d += blockDim.x) {
    s_o[d] = 0.0f;
}
__syncthreads();
```

Each thread processes element $t, t + \text{blockDim.x}, t + 2 \cdot \text{blockDim.x}, \ldots$ where $t = \text{threadIdx.x}$.

1.3 Tiled Flash Attention Algorithm

1.3.1 Main Loop Over KV Tiles

In this stage, the kernel iteratively processes the key–value (KV) sequence in fixed-size tiles to compute attention in a memory-efficient manner. Following the pseudo code in Figure. 9, we divided the implementation of the main loop into 10 different steps

```
For each KV tile in range [0, T) with causal masking:
```

```
  Step 1. Load K_tile and V_tile into shared memory
  Step 2. Compute S = Q @ K_tile^T (scores matrix: 8 x 64)
  Step 3. Apply causal mask: S[i,j] = -INF if position_j > position_i
  Step 4. Find block_max = max(S) per query row.
  Step 5. Compute and update max m_new = max(m_old, block_max)
  Step 6. Compute P = exp(S - m_new)
  Step 7. Rescale previous output: O = exp(m_old - m_new) * O
  Step 8. Compute: O += P @ V_tile
  Step 9. Update: d += sum(P, axis=1) for each query row
Step 10. Normalize output O using d and write to host
```

For each tile, the block loads the corresponding K and V vectors into shared memory, computes the partial score matrix ($S = QK^\top$), applies causal masking, and updates the running softmax statistics ($m_{\text{old}}, m_{\text{new}}, d$). The tile's contribution to the output is accumulated using the stable softmax formulation, ensuring numerical robustness even for long sequences.

```

for (int tile = 0; tile < num_tiles; tile++) {
    // Step 1: Load K and V tiles
    for (int kv_idx = 0; kv_idx < KV_TILE_SIZE; kv_idx++) {
        int kv_offset = tile * KV_TILE_SIZE + kv_idx;
        if (kv_offset < T) {
            for (int d = tid; d < D; d += blockDim.x) {
                // Load from global memory, keep as float for TF32 WMMA
                s_K[kv_idx*D + d] = K[batch_head_offset + kv_offset*D + d];
                s_V[kv_idx*D + d] = V[batch_head_offset + kv_offset*D + d];
            }
        } else {
            for (int d = tid; d < D; d += blockDim.x) {
                s_K[kv_idx*D + d] = 0.0f;
                s_V[kv_idx*D + d] = 0.0f;
            }
        }
    }
    __syncthreads();

    // Step 2: Compute Q @ K^T using WMMA
    compute_qk_wmma(s_qK, s_q, s_K, D, tid);

    // Step 3: Apply softmax scaling and causal masking
    for (int qk_idx = tid; qk_idx < KV_TILE_SIZE * Q_TILE_SIZE; qk_idx += blockDim.x)
        {
        int kv_idx = qk_idx % KV_TILE_SIZE;
        int q_idx = qk_idx / KV_TILE_SIZE;
        int q_offset = q_tile_offset + q_idx;
        int kv_offset = tile * KV_TILE_SIZE + kv_idx;

        if (q_offset >= T || kv_offset >= T || kv_offset > q_offset) {
            s_qK[q_idx*KV_TILE_SIZE + kv_idx] = -INFINITY;
        } else {
            s_qK[q_idx*KV_TILE_SIZE + kv_idx] *= softmax_scale;
        }
    }
    __syncthreads();

    // Step 4 Find block maximum
    for (int q_idx = tid; q_idx < Q_TILE_SIZE; q_idx += blockDim.x) {
        float block_max = -INFINITY;
        for (int kv_idx = 0; kv_idx < KV_TILE_SIZE; kv_idx++) {
            block_max = fmaxf(block_max, s_qK[q_idx*KV_TILE_SIZE + kv_idx]);
        }
        s_block_max[q_idx] = block_max;
        s_prev_max[q_idx] = s_row_max[q_idx];
    }
```

```cpp
    __syncthreads();

    // Step 5: Update global maximum
    for (int q_idx = tid; q_idx < Q_TILE_SIZE; q_idx += blockDim.x) {
        float new_max = fmaxf(s_block_max[q_idx], s_row_max[q_idx]);
        s_row_max[q_idx] = new_max;  // m_new
    }
    __syncthreads();

    // Step 6: Compute exp(S - m_new) with stability
    for (int qk_idx = tid; qk_idx < Q_TILE_SIZE*KV_TILE_SIZE; qk_idx += blockDim.x) {
        int kv_idx = qk_idx % KV_TILE_SIZE;
        int q_idx = qk_idx / KV_TILE_SIZE;
        // exp(s_{i,j} - m_new)
        s_qK[q_idx*KV_TILE_SIZE + kv_idx] =
            expf(s_qK[q_idx*KV_TILE_SIZE + kv_idx] - s_row_max[q_idx]);
    }
    __syncthreads();

    // Step 7 :Rescale previous output
    for (int qd_idx = tid; qd_idx < Q_TILE_SIZE * D; qd_idx += blockDim.x) {
        int q_idx = qd_idx / D;
        // O_old *= exp(m_old - m_new)
        float scale = expf(s_prev_max[q_idx] - s_row_max[q_idx]);
        s_o[qd_idx] = s_o[qd_idx] * scale;
    }
    __syncthreads();

    // Step 8: Compute: O += P @ V_tile
    compute_pv_wmma(s_o, s_qK, s_V, D, tid);


    // Step 9: Update softmax denominator for each query row
    for (int q_idx = tid; q_idx < Q_TILE_SIZE; q_idx += blockDim.x) {
        float block_sum_exp = 0.0f;
        for (int kv_idx = 0; kv_idx < KV_TILE_SIZE; kv_idx++) {
            block_sum_exp += s_qK[q_idx*KV_TILE_SIZE + kv_idx];
        }
        float scale = expf(s_prev_max[q_idx] - s_row_max[q_idx]);
        // d_new = d_old * exp(m_old - m_new) + sum(P)
        s_sum_exp[q_idx] = s_sum_exp[q_idx] * scale + block_sum_exp;
    }
    __syncthreads();
}

//Step 10: Normalize and write output
for (int qd_idx = tid; qd_idx < Q_TILE_SIZE * D; qd_idx += blockDim.x) {
    int q_idx = qd_idx / D;
    int d_idx = qd_idx % D;
    int q_offset = q_tile_offset + q_idx;
    if (q_offset < T) {
        float sum_exp_inv = 1.0f / s_sum_exp[q_idx];
        O[batch_head_offset + q_offset * D + d_idx] = s_o[qd_idx] * sum_exp_inv;  // O_i / d_i
    }
}
```

1.3.2 Q@K Computation with TF32 WMMA

The `compute_qk_wmma` device function computes $Q \times K^T$ using TensorFloat-32 (TF32) precision with tensor cores. It operates on the Q tile (Q_TILE_SIZE tokens $\times$ D dimensions) and KV tiles (KV_TILE_SIZE tokens $\times$ D dimensions), producing a 16×64 score matrix. The output is stored into `s_qK` shared memory for further reused in later steps.

```cpp
__device__ void compute_qk_wmma(
    float* s_qK,              // Output: 16 x 64 scores (float)
    const float* s_q,         // Input: 16 x D (float)
    const float* s_K,         // Input: 64 x D (float)
    int D,
    int tid
) {
    int warp_id = tid / WARP_SIZE;

    // TF32 requires D % 8 == 0 (WMMA_K = 8)
    if (D % WMMA_K == 0 && KV_TILE_SIZE % WMMA_N == 0 && Q_TILE_SIZE % WMMA_M == 0) {
        int num_warps = blockDim.x / WARP_SIZE;
        int num_k_tiles = KV_TILE_SIZE / WMMA_N;  // 64 / 16 = 4

        for (int k_tile = warp_id; k_tile < num_k_tiles; k_tile += num_warps) {
            int k_row_offset = k_tile * WMMA_N;

            // Declare TF32 WMMA fragments (hardware registers)
            wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K,
                wmma::precision::tf32, wmma::row_major> q_frag;
            wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K,
                wmma::precision::tf32, wmma::col_major> k_frag;
            wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;

            wmma::fill_fragment(acc_frag, 0.0f);

            // Iterate over depth dimension in chunks of WMMA_K (8)
            for (int d = 0; d < D; d += WMMA_K) {
                // Load Q tile (16x8) from s_q
                wmma::load_matrix_sync(q_frag, s_q + d, D);

                // Load K tile (16x8) from s_K in column-major
                wmma::load_matrix_sync(k_frag, s_K + k_row_offset * D + d, D);

                // Compute and accumulate: acc += q @ k^T
                wmma::mma_sync(acc_frag, q_frag, k_frag, acc_frag);
            }

            // Store result to shared memory
            wmma::store_matrix_sync(
                s_qK + k_row_offset,
                acc_frag,
                KV_TILE_SIZE,
                wmma::mem_row_major
            );
        }
    } else {
        // Fallback: scalar computation if dimensions not aligned
        for (int qk_idx = tid; qk_idx < KV_TILE_SIZE * Q_TILE_SIZE; qk_idx += blockDim.x) {
            int kv_idx = qk_idx % KV_TILE_SIZE;
            int q_idx = qk_idx / KV_TILE_SIZE;

            float qk = 0.0f;
            for (int d = 0; d < D; d++) {
                qk += s_q[q_idx * D + d] * s_K[kv_idx * D + d];
            }
            s_qK[q_idx * KV_TILE_SIZE + kv_idx] = qk;
        }
    }
    __syncthreads();
}
```

When the tensor-core path cannot be used—typically because the dimensions fail to satisfy the WMMA alignment constraints (D must be divisible by 8, and both Q_TILE_SIZE and KV_TILE_SIZE must be multiples of 16)—the kernel falls back to a standard scalar dot-product implementation. In this fallback mode, each thread computes individual QK scores by iterating over the full depth dimension (D), multiplying corresponding elements of the query and key vectors and accumulating the result in FP32. Although slower than TF32 WMMA, this path guarantees correctness for arbitrary tensor shapes and serves as a robust compatibility mode for non-aligned configurations.

1.3.3 P@V Computation with TF32 WMMA

The same idea goes for P@V operation. The `compute_pv_wmma` device function computes $P \times V$ where P is the attention probability matrix (16×64) and V is the value matrix ($64 \times D$), producing output of size $16 \times D$.

```
__device__ void compute_pv_wmma(
    float* s_o,
    const float* s_qK,      // 16 x 64 attention probabilities (float)
    const float* s_V,       // 64 x D value matrix (float)
    int D, int tid
) {
    int warp_id = tid / WARP_SIZE;
    int num_warps = blockDim.x / WARP_SIZE;

    // P @ V: [Q_TILE_SIZE, KV_TILE_SIZE] @ [KV_TILE_SIZE, D]
    if (D % WMMA_N == 0 && KV_TILE_SIZE % WMMA_K == 0 && Q_TILE_SIZE % WMMA_M == 0) {
        int num_d_tiles = D / WMMA_N;

        for (int d_tile = warp_id; d_tile < num_d_tiles; d_tile += num_warps) {
            int d_offset = d_tile * WMMA_N;

            wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K,
                wmma::precision::tf32, wmma::row_major> p_frag;
            wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K,
                wmma::precision::tf32, wmma::row_major> v_frag;
            wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;

            // Load existing output for accumulation
            wmma::load_matrix_sync(acc_frag, s_o + d_offset, D, wmma::mem_row_major);

            // Loop over KV dimension in chunks of WMMA_K (8)
            for (int kv = 0; kv < KV_TILE_SIZE; kv += WMMA_K) {
                wmma::load_matrix_sync(p_frag, s_qK + kv, KV_TILE_SIZE);
                wmma::load_matrix_sync(v_frag, s_V + kv * D + d_offset, D);
                wmma::mma_sync(acc_frag, p_frag, v_frag, acc_frag);
            }

            wmma::store_matrix_sync(s_o + d_offset, acc_frag, D, wmma::mem_row_major);
        }
    } else {
        // Fallback: scalar computation
        int lane_id = tid % WARP_SIZE;
        for (int d_start = warp_id * 16; d_start < D; d_start += num_warps * 16) {
            int d_end = min(d_start + 16, D);
            for (int q_row = lane_id; q_row < Q_TILE_SIZE; q_row += WARP_SIZE) {
                for (int d_col = 0; d_col < (d_end - d_start); d_col++) {
                    float acc = 0.0f;
                    for (int kv_idx = 0; kv_idx < KV_TILE_SIZE; kv_idx++) {
                        float p_val = s_qK[q_row * KV_TILE_SIZE + kv_idx];
                        float v_val = s_V[kv_idx * D + d_start + d_col];
                        acc += p_val * v_val;
                    }
                    s_o[q_row * D + d_start + d_col] += acc;
                }
            }
        }
    }
```

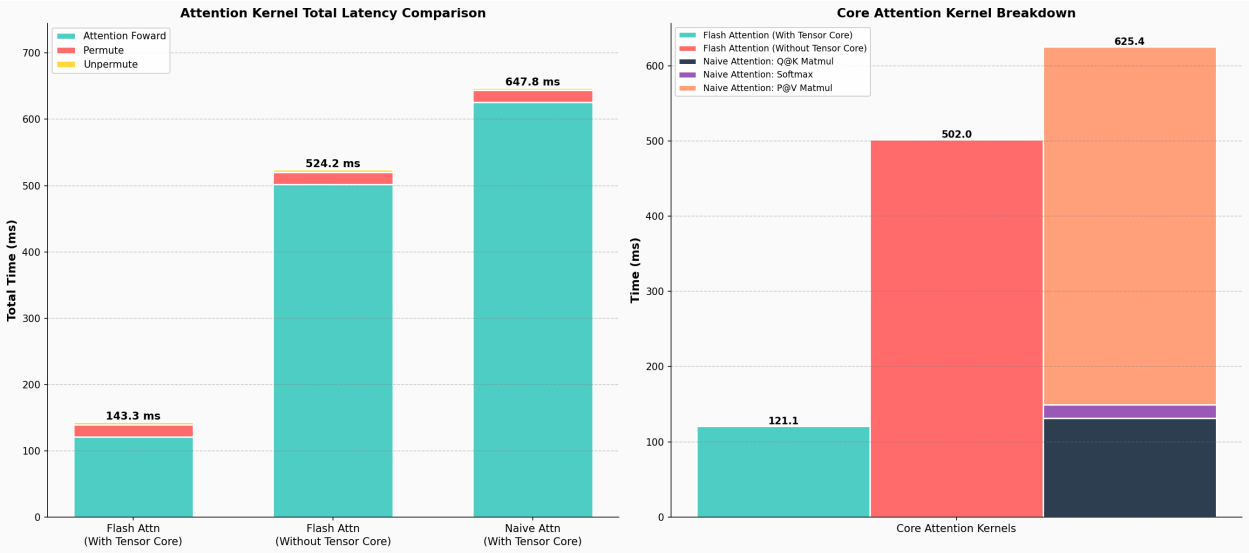

Figure 2: Attention Nsight System Profiling Results

```
    __syncthreads();
}
```

Same as previous kernel for QK^T, one noticeable limitation of this implementation for PV is that it currently cannot use Tensor Cores when the head dimension is not divisible by $WMMA_N = 16$ (or $WMMA_K = 8$ in QK^T case). Future optimization efforts will focus on extending Tensor Core support to arbitrary head sizes.

1.4 Comparison with previous naive Attention implementation

To test the ability to process long sequences, each of these kernels are lauch to process input of shape: $B \times T \times NH \times D$, where $B = 4$ is the batch size, $T = 4096$ is the sequence length, $NH = 2$ is the number of heads, and D is the hidden dimesion. To compare performance of Attention kernels, we perform Nsys analysis on three type of kernels:

- The Naive Attention, which launches three different kernels (i.e, QK^T , PV GEMMs and Softmax). Tensor core are used in GEMMs kernels
- The Flash Attention kernel without using Tensor Core
- The Flash Attention kernel with Tensor Core as described in previous section.

As shown in Figure 2, based on the Nsight System profiling results, even the Flash Attention kernel without Tensor Cores outperforms the naïve attention implementation that uses Tensor Cores (about $\times 1.25$ faster). When Tensor Cores are enabled, the Flash Attention kernel achieves an even larger speedup, reaching approximately $\times 5.16$ faster overall.

1.5 Further Analysis Flash Attention With and Without Tensor Cores

This section further analyzes how the Flash Attention kernel behaves under two execution modes: a standard CUDA-core implementation and a Tensor Core–accelerated implementation. As shown in figure. 3, The non-TC kernel achieves high memory utilization ($\sim$82%) but only $\sim$10% compute utilization, confirming it is severely memory-bound. The kernel issues thousands of scalar memory instructions per tile, saturating the memory subsystem while CUDA cores wait for data. Conversely, the TC kernel shows lower memory utilization ($\sim$33%) and better compute utilization (18%). To better analyse the performance bottleneck, we can look at metrics for warp stall state (figure 4) which reveal distinct bottleneck regimes for each kernel.

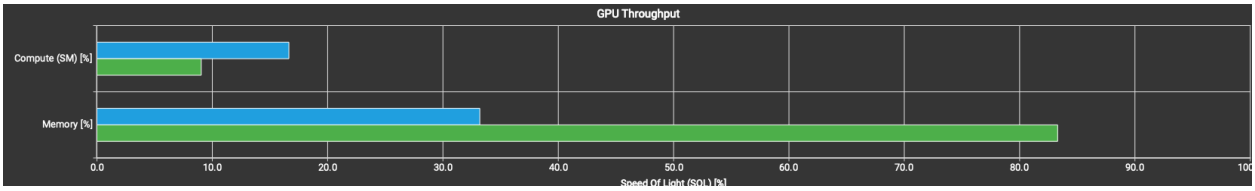

Figure 3: High-level overview of compute and memory throughput for the Flash Attention kernel with Tensor Cores (blue) and without Tensor Cores (green).

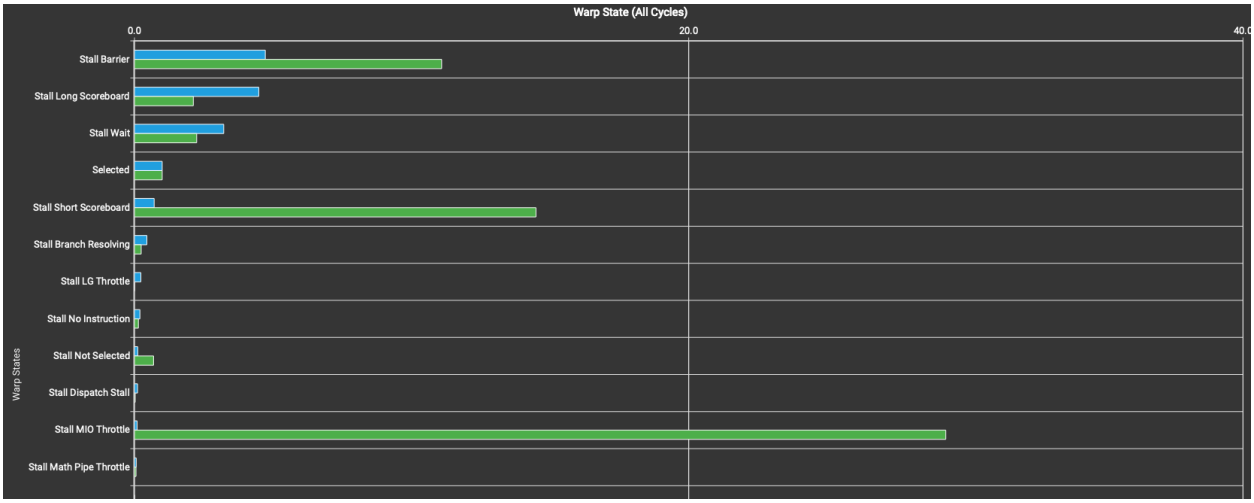

Figure 4: Warp stall state distribution for the non-Tensor Core (green bar) and Tensor Core Kernel (blue bar) implementations.

Non-Tensor Core Kernel: Memory-Limited Behavior

MIO Throttle . As shown in figure. 4 The dominant performance limiter in the non-TC kernel is memory instruction queue saturation. This might stem from the QK^T and PV phases, where each thread performs a scalar dot product over the full depth dimension:

```
for (int d = 0; d < D; d++) {
    qk += s_q[q_idx * D + d] * s_K[kv_idx * D + d];
}
```

Each iteration issues two shared-memory loads. With 128 threads computing 512 dot products per tile (16×32), the memory instruction queue becomes overloaded, causing long MIO Throttle stalls as the GPU cannot retire memory instructions as fast as they are generated.

Short Scoreboard As expected a secondary bottleneck arises from shared-memory latency. The strided access pattern `s_K[kv_idx * D + d]`—especially for common dimensions such as $D = 64$—causes shared-memory bank conflicts. These conflicts serialize loads across threads, leading to recurring Short Scoreboard stalls.

Overall, the non-TC kernel is heavily memory-bound, with performance dominated by shared-memory pressure and instruction saturation rather than compute throughput.

Tensor Core Kernel: Synchronization-Limited Behavior

Stall Barrier . As shown in figure. 4, when Tensor Cores are enabled, the major bottleneck shifts from memory traffic to synchronization. Many warps have to wait at `__syncthreads()` barriers. The computation becomes so fast that tile boundary synchronization emerges as the new limiting factor.

Long Scoreboard . A suprising bottleneck is the long Long scoreboard stall which arise from waiting for global memory load/store access. In our kernel most of the load store are perform on the shared mem so this might stem

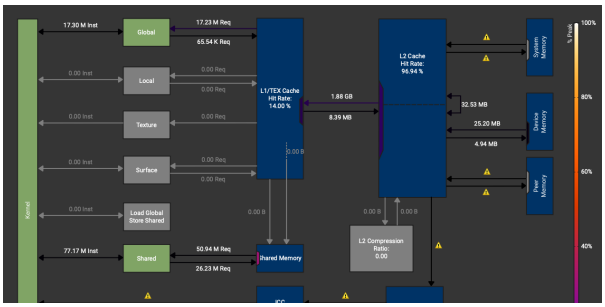

Figure 5: Flash Attn kernel using Tensor Core

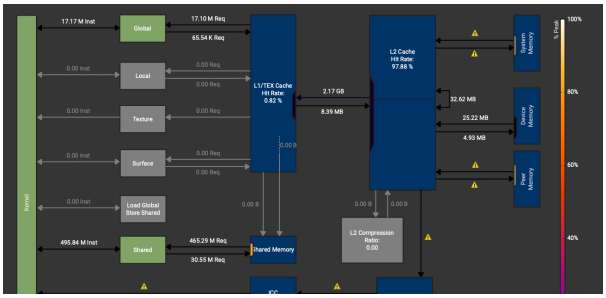

Figure 6: Flash Attn kernel without using Tensor Core

from the fact that the Tensor Core path uses large tiles and many WMMA fragments, the register and shared-memory footprint per block is high, which limits occupancy and reduces the number of independent warps available to hide these long-latency accesses. Consequently, the remaining global (and spilled local) memory transactions cannot be fully overlapped with computation and show up in Nsight as Long Scoreboard stalls, even though the majority of memory operations inside the tile are served from shared memory.

WHY MIO THROTTLE DISAPPEARS

The key improvement in the Tensor Core version is the elimination of scalar FMAs. A single WMMA instruction ($16 \times 16 \times 8$, TF32) performs 2048 fused-multiply–add operations in hardware. This replaces thousands of scalar operations that would otherwise issue separate shared-memory loads, dramatically reducing memory instruction pressure. As shown in figures 5 6, where the non-tensor core version have more than 400 millions shared memory requests, the tensor core version only need approx 50 millions memory requests. As a result, MIO Throttle stalls vanish almost entirely—the kernel is no longer memory-instruction–bound, but instead synchronization-bound. The Tensor Core kernel successfully eliminated memory instruction bottlenecks but exposed synchronization as the new limiting factor.

2 REQUIREMENT 5: CONFIGURATION SWEEP

In this report, due to limited time and resources, we focus primarily on configuration choices for the FlashAttention kernel. Matrix multiplication is already well optimized and efficiently handled by cuBLAS, whereas FlashAttention is one of the most critical custom kernels and accounts for a large fraction of the total inference time.

2.1 FLASH ATTENTION

2.1.1 Q TILE SIZE & KV TILE SIZE

The performance of our FlashAttention kernel depends critically on two tiling parameters: the query tile size Q_TILE_SIZE and the key-value tile size KV_TILE_SIZE. These parameters control how the attention computation is partitioned across thread blocks and streamed through shared memory.

- **Query Tile Size (Q_TILE_SIZE).** This parameter determines the number of query tokens processed by each thread block. The grid launches $\lceil T/Q_TILE_SIZE \rceil$ blocks along the sequence dimension, where T is the sequence length. A larger Q_TILE_SIZE improves reuse of the streamed key-value data across more output rows, increasing arithmetic intensity. However, it also increases shared memory consumption for the query buffer and output accumulator, potentially reducing occupancy. Furthermore, overly large Q_TILE_SIZE values reduce block-level parallelism, which may underutilize streaming multiprocessors for shorter sequences.
- **Key-Value Tile Size (KV_TILE_SIZE).** This parameter controls the chunk size for streaming keys and values through shared memory. The kernel iterates $\lceil T/KV_TILE_SIZE \rceil$ times, loading a fresh $KV_TILE_SIZE \times d$ tile of K and V each iteration. Larger KV_TILE_SIZE reduces the number of global memory load phases and synchronization barriers, while also enabling larger matrix multiplications that better utilize tensor core throughput. However, the shared memory footprint grows as $2 \times KV_TILE_SIZE \times d$ for the key-value buffers plus $Q_TILE_SIZE \times KV_TILE_SIZE$ for the attention score matrix, making this the dominant contributor to shared memory pressure.

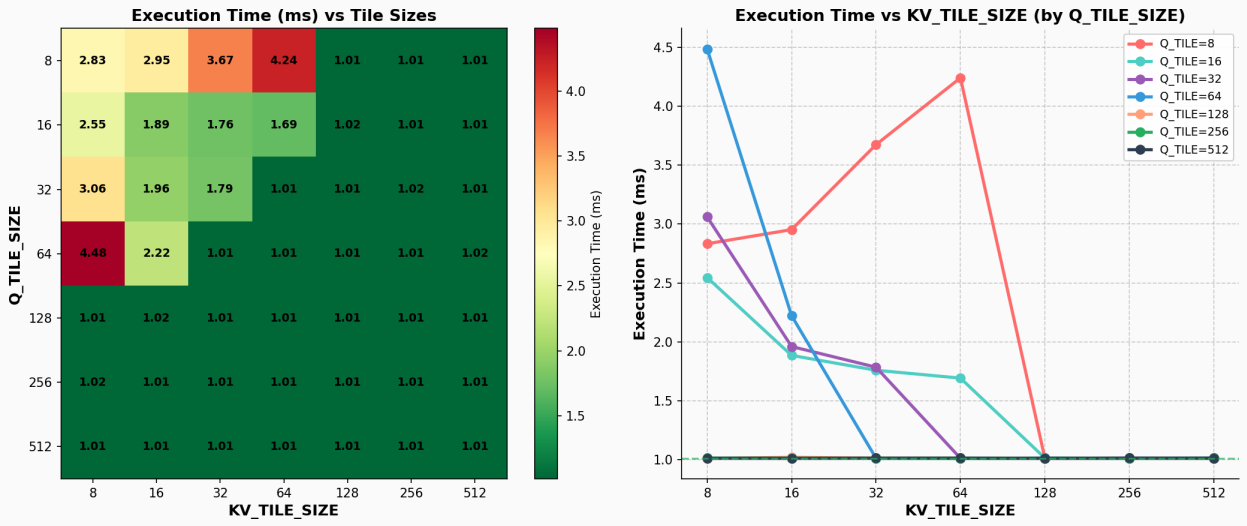

Figure 7: Q tile size vs KV tile size profiling results

- **Configuration Space.** The optimal tile sizes depend on multiple factors: head dimension d, sequence length T, GPU shared memory capacity, and the compute-to-memory ratio of the target architecture. No single configuration performs optimally across all settings. We therefore sweep over valid tile size combinations satisfying the tensor core alignment constraints ($Q_TILE_SIZE, KV_TILE_SIZE$) and the shared memory budget, selecting the configuration that minimizes end-to-end latency for each problem size.

Figure 7 presents the execution time across different combinations of Q_TILE_SIZE and KV_TILE_SIZE. Several key observations emerge from this sweep. **Small Tiles Incur High Overhead.** The top-left region of the heatmap shows that small tile sizes in both dimensions lead to poor performance, with execution times exceeding 4ms. This occurs because small tiles result in many inner loop iterations, each incurring synchronization barriers and global memory load phases. Another reason might be that tile size that does not take avantage of Tensor Core, which might significantly slow down the kernel.

- **Large Tiles Saturate Performance.** Configurations with $Q_TILE_SIZE \geq 128$ or $KV_TILE_SIZE \geq 128$ consistently achieve near-optimal execution times around 1.01ms, regardless of the other parameter. This suggests the kernel becomes compute-bound once either tile dimension is sufficiently large, amortizing memory access and synchronization costs. One suprising thing to notice is that compute-bound happen even when KV_TILE_SIZE is lower than 16, which lead to unsuitable matrix sizes for tensor core operations.

- **Asymmetric Sensitivity.** The line plot reveals that large Q_TILE_SIZE values (128, 256, 512) maintain stable performance across all KV_TILE_SIZE settings, while small Q_TILE_SIZE values exhibit high sensitivity to KV_TILE_SIZE. This asymmetry arises because increasing Q_TILE_SIZE reduces the number of thread blocks, allowing each block to better amortize the streaming cost of keys and values. In contrast, small Q_TILE_SIZE launches many blocks that each must independently iterate through the KV tiles.

- **Practical Recommendation.** Based on these results, we select $Q_TILE_SIZE = 128$ and $KV_TILE_SIZE = 64$ as our default configuration, balancing shared memory usage against performance. Larger tiles provide no additional benefit while increasing register and shared memory pressure.

2.1.2 Thread Count and Unroll Factor Analysis

Now given the best configuration from previous sweep, we continue to analyze the next set of parameters which are the number of threads in each block and the unroll factor .Figure 8 presents the execution time across different combinations of thread count and loop unroll factor. The results reveal notably different characteristics compared to the tile size sweep.

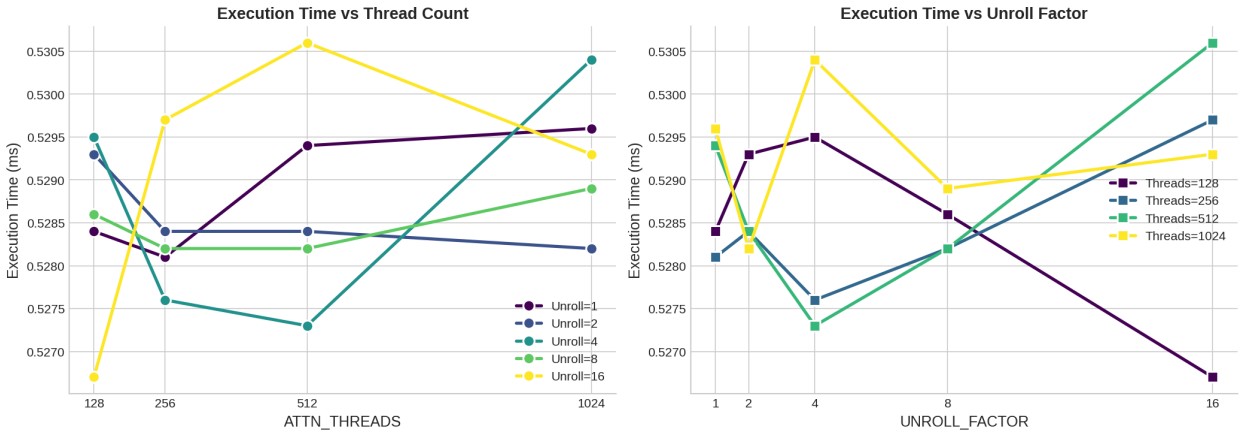

Figure 8: Threads and Loop unroll

- **Minimal Performance Variation.** In general execution time range across all configurations is narrow, spanning only 0.5267–0.5306 ms (less than 0.8% variation). making these parameters less critical than tile sizes for overall performance.
- **Unroll Behavior.** The left panel shows that aggressive unrolling (factor of 16) exhibits the highest variance across thread counts. At 128 threads, Unroll=16 achieves the fastest execution (0.5267 ms), but at 512 threads it produces the slowest (0.5306 ms). This divergence arises from register pressure: aggressive unrolling with large thread blocks exceeds the register file capacity, causing spills to local memory.
- **Thread Count Sensitivity.** The right panel reveals that 128 threads (purple) benefits from increasing unroll factors, with execution time dropping from 0.5284 ms at Unroll=1 to 0.5267 ms at Unroll=16. In contrast, 512 threads (green) and 1024 threads (yellow) show different behavior, with performance degrading at high unroll factors. This indicates that smaller thread blocks provide more headroom for register-intensive optimizations.

Practical Recommendation. Given the minimal performance difference and the instability of extreme configurations, we select 128 threads with an unroll factor of 16 as our optimal configuration (0.5267 ms).

3 Requirement 6: Constant Memory

3.1 Targets

Since the constant memory size on the A40 is only 64 KB, we must carefully consider which data we use as candidates to store in constant memory. Table 3 provides potential candidates for constant memory usage and why we think they should or should not be cacded in constant memory. As summarized in Table 3, we focus on layer normalization, where each layer's scale and bias parameters are small enough to fit within the 64 KB constant memory. The implementation is modified to store and load the layer norm weights and biases in `__constant__` memory.

Listing 1: Modifed Layer norm with constant memory

```
__constant__ float const_ln_weight[8192];
__constant__ float const_ln_bias[8192];

__global__ void layernorm_forward_kernel(float *out, float *mean, float *rstd, const
    float *inp, int B, int T, int C) {
    // layer norm logic
    ....
    // Normalize and scale + shift
    for (int c = threadIdx.x; c < C; c += blockDim.x) {
        float val = inp[token_start + c];
        float norm = (val - m) * r;
        out[token_start + c] = norm * const_ln_weight[c] + const_ln_bias[c];
    }
}
```

Candidates	Size Each Layer	Suitable?	Reason
Token Embeddings (`wte`)	154 MB	No	Scattered, token-dependent lookups. The embedding table's memory is much larger than A40's 64KB constant memory. Embedding lookups also vary across tokens, preventing warp-wide broadcast benefits. Global memory + L2 cache are the only feasible locations.
Position Embeddings (`wpe`)	3 MB	No	Still too large for constant memory. Although positional encodings are reused. Constant memory usage is unnecessary and too small.
QKV Weights (`qkvw`)	7.08 MB	No	These weights are larger than constant cache size.
Attn Projection (`attprojw`)	2.3 MB	No	Larger than constant size. Better stored in Global Mem
FC Weights (`fcw`)	9.4MB	No	Larger than constant cache size. Better stored in Global Mem
FC Projection (`fcprojw`)	9.4MB	No	Same size as FC weights and far too large. Better stored in Global Mem
Layer Norm's Weight	6 KB	Potential	Small (768 parameters each layer)
Layer Norm's Bias	6 KB	Potential	Same small size (768 parameters each layer)

Table 3: Candidates for constant memory and justification for exclusion.

Metric	Without `__constant__`	With `__constant__`
Duration (μs)	0.04	0.10 ($\times 2.5$ slower)
Compute Throughput (%)	51	75
Memory Throughput (%)	82	23

Table 4: Impact of `__constant__` on constant kernel performance.

3.2 Profiling results

Given the results in Table 4, we observe that while compute throughput increases when using `__constant__` memory, overall performance degrades due to significantly worse effective memory behavior. Although constant memory can provide low-latency access through warp-level broadcast, this benefit is only realized when all threads in a warp access the same address. In the layer normalization kernel, each thread accesses a distinct index of the weight and bias vectors (indexed by the channel dimension), resulting in serialized constant memory accesses rather than broadcasts. Consequently, constant memory requests are effectively serviced one-by-one, increasing access latency and reducing effective memory throughput. This mismatch between the access pattern and the intended usage of `__constant__` memory explains why higher compute utilization does not translate into lower execution time, ultimately making the constant-memory variant slower than the baseline.

4 Requirement 7: Using `__restrict__`

4.1 Overview

The `__restrict__` keyword is a type qualifier that informs the compiler that a pointer is the sole means of accessing the memory it references during its lifetime. This guarantee of no pointer aliasing enables the compiler to perform more aggressive optimizations, including:

- keeping values in registers instead of reloading from memory,
- reordering loads and stores more freely,
- and reducing the need for conservative memory barriers.

4.2 PERMUTE KERNEL ANALYSIS

We implemented `__restrict__` keyword in many kernels, but we chose to evaluate a *permute* kernel that reorganizes the input tensor from shape $(B, T, 3, N_H, H_S)$ into three separate Q, K, and V tensors, each with shape (B, N_H, T, H_S). This kernel's operation is memory-bound and involves strided global memory accesses. making it a strong example for this optimization analysis.

4.2.1 PERFORMANCE SUMMARY

Metric	Baseline	Optimized	Change
Duration (μs)	23.71	9.89	-58.3% ($2.40\times$ speedup)
Compute Throughput (%)	0.36	0.86	$+140.3\%$
Memory Throughput (%)	3.03	4.24	$+39.77\%$
Registers / Thread	48	40	-16.7%

Table 5: Impact of `__restrict__` on permute kernel performance.

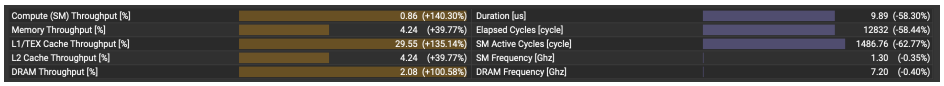

Figure 9: Permute Kernel Comparison

The number of registers per thread decreased from 48 to 40 (a 16.7% reduction). Without `__restrict__`, the compiler must assume that any two pointer arguments may alias, forcing it to:

- Reload values from memory after stores, in case a write through one pointer has modified data previously loaded through another (potentially aliasing) pointer.
- Keep additional live values in registers to handle worst-case aliasing.

Once aliasing ambiguity is removed, the compiler can safely eliminate these redundant reloads and shrink the register footprint. Memory throughput increased by 39.9%, indicating more efficient utilization of the memory subsystem. This comes largely from reducing redundant loads that were previously necessary to satisfy conservative alias assumptions.

Compute throughput improved by 138.9% (from 0.36 to 0.86), suggesting better instruction-level parallelism (ILP). When the compiler knows that stores cannot affect subsequent loads through aliasing, it can more aggressively reorder instructions to hide memory latency and keep arithmetic units busy. Finally, we observed approximately a 13.64% reduction in the number of branch instructions. This is because that the compiler no longer needing certain aliasing conditional checks and being able to apply more aggressive loop unrolling once the pointer relationships are known.

4.2.2 CYCLE-LEVEL ACTIVITY

We also examined SM and memory subsystem activity to understand the impact of `__restrict__` at the micro-architectural level.

Metric	Change
SM Active Cycles (Avg)	-62.77%
SMSP Active Cycles (Avg)	-63.21%
L1 Active Cycles (Avg)	-62.77%
L2 Active Cycles (Avg)	-61.67%
DRAM Active Cycles (Avg)	-16.69%
Total SM Elapsed Cycles	-59.38%

Table 6: Change in active cycles with `__restrict__` enabled.

All levels of the memory hierarchy show a large reduction in active cycles. This indicates that the `__restrict__` qualifier allowed the compiler to:

- Remove redundant load/store instructions,
- And perform better scheduling between memory operations.

Occupancy remains largely unchanged between the baseline and optimized kernel. This is expected: `__restrict__` is a *compiler-side* optimization that primarily improves generated instruction sequences and memory behavior, rather than changing kernel launch configuration or hardware resource allocation directly. The speedup comes from doing less work per thread, not from launching more threads.

4.3 Takeaways

Using `__restrict__` on the permute kernel delivered:

- A $2.40\times$ speedup in kernel duration,
- Higher compute and memory throughput,
- Lower register usage and fewer branch instructions,
- And significantly fewer active cycles across SM, L1, L2, and DRAM.

Because the permute operation is heavily memory-bound and uses multiple pointers which conceptually never alias, it is an ideal target for `__restrict__`. This optimization brings meaningful performance gains by enabling the compiler to generate more parallel code.

5 Requirement 8: Local Attention

5.1 Implementation

For local attention implementation, we modify the flash attention kernel as follow:

- In step 1: Key and value vectors are loaded only if they fall inside the window:$\max(0, q_{\text{tile}} - W) \leq k < q_{\text{tile}} + Q_{\text{TILE_SIZE}}$. All other entries are zero-filled, reducing shared memory traffic.
- In step 3: After computing QK^T Apply mask for every token that:

$$\text{mask}(q,k) = \begin{cases} -\infty, & k > q \ \vee \ k < q - W, \\ Q_q \cdot K_k \cdot \text{scale}, & \text{otherwise.} \end{cases}$$

```
    // For a tile of queries [q_tile_offset, q_tile_offset + Q_TILE_SIZE),
    // the union of all windows is [max(0, q_tile_offset - WINDOW_SIZE), q_tile_offset
        + Q_TILE_SIZE)
    int window_start = max(0, q_tile_offset - WINDOW_SIZE);
    int window_end = q_tile_offset + Q_TILE_SIZE;
    int start_tile = window_start / KV_TILE_SIZE;
    int end_tile = CEIL_DIV(window_end, KV_TILE_SIZE);
    // Step 1: Load K and V tiles

    for (int tile = start_tile; tile < end_tile; tile++) {
        for (int kv_idx = 0; kv_idx < KV_TILE_SIZE; kv_idx++) {
            int kv_offset = tile * KV_TILE_SIZE + kv_idx;
            if (kv_offset < T && kv_offset >= window_start && kv_offset < window_end)
                {
                for (int d = tid; d < D; d += blockDim.x) {
                    s_K[kv_idx * D + d] = K[batch_head_offset + kv_offset * D + d];
                    s_V[kv_idx * D + d] = V[batch_head_offset + kv_offset * D + d];
                }
            } else {
                for (int d = tid; d < D; d += blockDim.x) {
                    s_K[kv_idx * D + d] = 0.0f;
                    s_V[kv_idx * D + d] = 0.0f;
```

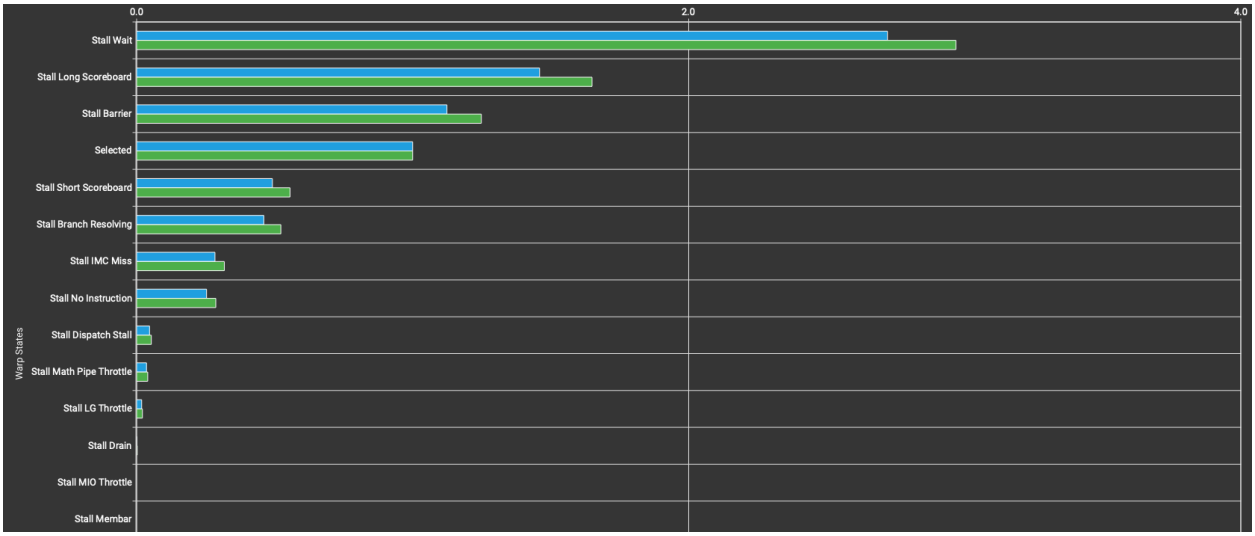

Figure 10: Warp Statistic of local Attention (blue) vs global Attention (green)

```
                }
            }
        }
        __syncthreads();

        // Step 2: Compute Q @ K^T using TF32 WMMA
        compute_qk_wmma(s_qK, s_q, s_K, D, tid);

        // Step 3: Apply softmax scaling, causal masking, and per-query window masking

        for (int qk_idx = tid; qk_idx < KV_TILE_SIZE * Q_TILE_SIZE; qk_idx += blockDim.x) {
            int kv_idx = qk_idx % KV_TILE_SIZE;
            int q_idx = qk_idx / KV_TILE_SIZE;
            int q_offset = q_tile_offset + q_idx;
            int kv_offset = tile * KV_TILE_SIZE + kv_idx;

            // For each query, the window is [max(0, q_offset - WINDOW_SIZE), q_offset]
            int query_window_start = max(0, q_offset - WINDOW_SIZE);

            if (q_offset >= T || kv_offset >= T || kv_offset > q_offset || kv_offset < query_window_start) {
                s_qK[q_idx * KV_TILE_SIZE + kv_idx] = -INFINITY;
            } else {
                s_qK[q_idx * KV_TILE_SIZE + kv_idx] *= softmax_scale;
            }
        }
        __syncthreads();

        // the rest of the kernel stay the same
```

5.2 Profiling Result

From Figure 10, we observe a clear reduction in the number of warps experiencing long scoreboard stalls, which indicate waiting on global or shared memory. This improvement arises because local attention avoids loading and computing tokens that lie outside the specified window, thereby reducing unnecessary memory accesses and associated stalls.

Method	Generation Length	Total Inference time (ms)	Token/Sec
Without local attention	50	317	157
Without local attention	500	6358	78
With local attention	50	**311**	**152**
With local attention	500	**5975**	**83**

Table 7: Speed comparison before and after applying local attention

In Table 7, we observe a consistent reduction in end-to-end latency with local attention, with the benefit becoming more pronounced at longer generation lengths, where pruning out-of-window tokens significantly reduces redundant memory accesses and synchronization overheads.

6 Requirement 9: Split-K

6.1 Target Identification: The Tail Effect

Standard Matrix Multiplication (GEMM) algorithms parallelize computation by tiling the output dimensions (M and N). A single thread block is responsible for computing a fixed tile of the output matrix. While efficient for large matrices, this approach suffers from the "Tail Effect" when the output dimensions are small relative to the GPU's available resources.

In our GPT-2 inference configuration ($B = 4, T = 64$), the activation matrix size is $M = B \times T = 256$. The inner dimension K (channels), however, is large ($4 \times C = 3072$) in the MLP layers. We identified the **MLP Output Projection** layer as a critical bottleneck for standard CUTLASS kernels:

- **Problem Dimensions:** $M = 256, N = 768, K = 3072$.
- **Baseline Tiling:** Using a standard tile size, the grid dimension is determined by $\lceil M/Tile_M \rceil \times \lceil N/Tile_N \rceil$. For these dimensions, this results in a very small number of thread blocks (e.g., 6 blocks).
- **Hardware Under-utilization:** With only ~6 thread blocks, a GPU with 84 Streaming Multiprocessors (SMs) operates at less than 8% occupancy. The remaining SMs are idle, waiting for the few active blocks to finish their long accumulation loops over $K = 3072$.

6.2 Implementation

To address this, we implemented the Split-K optimization using the CUTLASS `gemm_splitk_parallel` template. Split-K increases parallelism by splitting the summation dimension (K) into S slices. Each slice is computed by a separate thread block, calculating a partial sum. These partial sums are then reduced (summed) to produce the final output.

We modified `matmul.cuh` to dynamically select the Split-K kernel based on input dimensions. Our heuristic applies Split-K only when the problem size is insufficient to fill the GPU and the K-dimension is large enough to justify the reduction overhead.

```
// From matmul.cuh
// Heuristic: Use Split-K if output matrix is small but reduction dim K is large
bool use_split_k = (M * N < 1024 * 128) && (K >= 1024);

if (use_split_k) {
    // Split K into 4 slices to increase occupancy
    int split_slices = 4;
    status = run_gemm<CutlassGemmSplitK>(out, inp, weight, M, N, K, split_slices);
}
```

We configured the split factor $S = 4$. This effectively multiplies the number of thread blocks by 4, converting the deep serial accumulation over K into parallel work distributed across more SMs.

6.3 Performance Analysis

We profiled the MLP Output Projection layer comparing the baseline CUTLASS kernel against our Split-K implementation. The results demonstrate a large shift in how the GPU resources are utilized.

Metric	**Baseline**	**Split-K** ($S = 4$)	**Delta**
Duration	$78.69\mu s$	$58.59\mu s$	**-25.54%**
Cycles	102,596	76,257	-25.67%
SM Throughput	Low	High	**+168.28%**
Tensor Pipe Utilization	-	-	+34.66%
L2 Hit Rate	-	-	**+130.93%**
Memory Throughput	-	-	+41.72%

Table 8: Performance impact of Split-K optimization.

6.3.1 Throughput and Occupancy

The most dramatic improvement is the **168.28% increase in SM Throughput**. In the baseline configuration, the GPU was effectively starving; the few active SMs could not hide the latency of memory fetches or pipeline stalls. By splitting K, we flooded the GPU with enough thread blocks to hide these latencies. This is further supported by the **48.53% increase in SM Active Cycles**, confirming that more of the GPU was kept busy doing useful work during the kernel's execution.

6.3.2 Compute Characteristics

We observed a distinct shift in the compute pipeline profile:

- **Tensor Pipe Utilization (+34.66%):** The Split-K kernel was able to feed the Tensor Cores more consistently. The baseline likely suffered from pipeline bubbles where the Tensor Cores waited for data. With higher occupancy, the scheduler could switch to other warps, keeping the Tensor Cores saturated.
- **ALU Pipe Utilization (-41.54%):** Surprisingly, the utilization of the generic ALU pipe decreased. We hypothesize that the baseline kernel spent a larger proportion of its cycles on address calculations and loop overhead relative to useful math. By tightening the execution loops and saturating the Tensor pipe, the relative burden on the ALU pipe diminished.

6.3.3 Memory Hierarchy Behavior

Split-K is often criticized for increasing memory traffic (due to writing and reading partial sums). Our profiling confirms a **41.72% increase in Memory Throughput**, which aligns with the overhead of atomic reductions.

However, the **L2 Hit Rate increased by 130.93%**. This indicates that despite the higher total traffic, the access patterns were significantly more cache-friendly. Since the split slices share the same Input (A) and Weight (B) matrices (accessed at different offsets), the higher concurrency likely allowed these shared input tiles to remain resident in the L2 cache and be reused by multiple SMs simultaneously, whereas the serialized baseline might have evicted them between long-running iterations.

6.3.4 Conclusion

The Split-K optimization successfully converted a latency-bound "tail" problem into a throughput-bound workload. By sacrificing some total memory bandwidth (to store partial sums), we unlocked the GPU's idle compute resources. The trade-off yielded a **25% reduction in latency** and a massive improvement in hardware utilization, proving essential for low-batch inference scenarios.

7 Proposed Optimizations

7.1 Proposed Optimization 1: Targeted Kernel Fusion

7.1.1 Fused Residual and Layer Normalization

Implementation Strategy In the standard Transformer architecture, the Residual connection and Layer Normalization are performed sequentially. A baseline implementation typically launches two separate kernels: one element-wise kernel to compute $x \leftarrow x + r$ (saving the result to global memory) and a second reduction-heavy kernel to compute $\text{LayerNorm}(x)$.

This approach incurs significant memory overhead. The intermediate result of the residual addition is written to GPU global memory (DRAM) only to be immediately read back by the Layer Normalization kernel. This "round-trip" wastes memory bandwidth and incurs unnecessary kernel launch latency.

To address this, we implemented a **Fused Add-LayerNorm** kernel. This kernel performs the element-wise residual addition and immediately uses the result for the Layer Normalization statistics accumulation within the same thread, keeping the data in registers.

As shown in our implementation, we fuse the read, add, and accumulation steps into a single pass:

```
// From fused_add_layernorm.cuh
for (int i = threadIdx.x; i < C; i += blockDim.x) {
    int idx = offset + i;

    // 1. Load Data
    float r = residual_in[idx];
    float x = inp[idx];

    // 2. Compute Residual (Element-wise)
    float val = r + x;

    // 3. Store Residual Output (for next layer skip connection)
    residual_out[idx] = val;

    // 4. Accumulate Stats for LayerNorm (Reduction)
    lmean += val;
    lvar += val * val;
}
```

By keeping `val` in a register between step 3 and step 4, we eliminate the need to reload it from global memory for the normalization phase. The kernel then proceeds to use shared memory (`buf_mean`, `buf_var`) to perform a block-wide parallel reduction to compute the mean and standard deviation, finalizing the normalization and applying the affine transformation weights and bias.

Performance Analysis After profiling the fused implementation against the baseline sequential approach (Pure Residual kernel + Pure LayerNorm kernel), the results highlight the effectiveness of kernel fusion in "hiding" arithmetic operations behind expensive memory or synchronization barriers.

1. Latency Hiding and Speedup

The most distinct finding from our profiling is that the fused kernel executes in nearly the same time as the standalone Layer Normalization kernel, despite doing more work (the residual addition).

First Kernel Launch	Duration (μs)	Cycles
Pure Residual	3.10	3994
Pure LayerNorm	5.60	7239
Fused Add-LayerNorm	**5.70**	**7364**

Table 9: Latency comparison of baseline vs. fused kernels.

If executed sequentially, the total duration would be approximately $8.7\mu s$. The fused kernel completes in $5.70\mu s$, yielding a $1.52\times$ **speedup** for this sequence. This indicates that the cost of the element-wise residual addition is effectively "free" in the fused kernel. It is completely hidden by the latency of the Layer Normalization's memory transactions and reduction arithmetic.

2. Utilization and Throughput

The profiling data reveals a shift in hardware utilization that explains why the fusion is efficient.

- **SM Throughput:** The Pure Residual kernel had a very low SM throughput of **0.45%**. This confirms it was heavily latency-bound; the SMs were idling while waiting for simple read/write requests to complete.
- **Balanced Workload:** The Fused kernel achieves **1.86%** SM throughput, comparable to the Pure LayerNorm's **1.83%**.

This demonstrates that we have successfully shifted the workload. By merging the memory-bound residual operation into the compute-heavier LayerNorm context, we utilize the SMs more effectively. The idle cycles that previously existed during the residual kernel's execution are now filled with the arithmetic necessary for the LayerNorm statistics.

3. Pipe Utilization and Memory Efficiency

- **FMA Pipe:** The Fused kernel showed a slight increase in FMA (Fused Multiply-Add) pipe utilization compared to the Pure LayerNorm. This aligns with our expectation: the fusion introduces additional floating-point addition instructions ($val = r + x$) that must be processed by the math pipes.
- **Memory Throughput:** The fused kernel achieved a memory throughput **69% higher** than the standalone LayerNorm. This is a positive indicator of intensity: we are moving more data (reading residual, reading input, writing residual) per unit of time, but because we do it in a single launch without the intermediate global memory turnaround, the overall system memory bandwidth is used more productively.

Conclusion

The Fused Residual-LayerNorm optimization validates the principle of arithmetic intensity. The standalone residual addition was too lightweight to justify the overhead of a kernel launch and the latency of global memory access. By fusing it, we amortized these costs over the heavier workload of the Layer Normalization, achieving a $1.5\times$ speedup and significantly improving SM utilization.

7.1.2 Fused MatMul and GeLU

Implementation Strategy The second fusion target identified was the sequence of Matrix Multiplication, Bias Addition, and the GeLU activation function. In the baseline implementation, these are treated as distinct operations: a cuBLAS call for the GEMM ($X = W \cdot I$), followed by a kernel for bias addition ($X = X + b$), and finally a kernel for the activation ($Y = \text{GeLU}(X)$).

Our goal was to fuse these operations to reduce kernel launch overhead and improve memory locality. We implemented a wrapper function, `fused_matmul_gelu_forward`, which consolidates these operations into a single function call.

```
// From fused_matmul_gelu.cuh
void fused_matmul_gelu_forward(...) {
    // 1. Compute MatMul using cuBLAS
    cublasSgemm(..., out, ...);

    // 2. Apply Bias Addition
    if (bias != NULL) {
        fused_add_bias_kernel<<<...>>>(out, bias, ...);
    }

    // 3. Apply GeLU
    fused_gelu_kernel<<<...>>>(out, out, total_elements);
}
```

While this implementation simplifies the host code and ensures operations are enqueued on the stream efficiently, it relies on a "host-side" fusion strategy rather than a "device-side" fusion. We invoke standard cuBLAS kernels which must write their results to global memory before the subsequent bias and GeLU kernels can read them.

Performance Analysis We profiled the performance of the activation step in this "fused" sequence against the standalone GeLU kernel from the baseline. Because the Matrix Multiplication relies on the highly optimized cuBLAS library, we cannot inject the GeLU logic directly into the GEMM's accumulation loop without using advanced APIs like `CUTLASS`. Therefore, our analysis focuses on the behavior of the GeLU kernel when launched immediately following the GEMM in this consolidated sequence.

1. Memory and Cache Behavior

The most notable improvement in the fused sequence was in cache efficiency.

- **L1 Hit Rate:** The fused GeLU kernel exhibited an **L1 hit rate 300% higher** than the standalone GeLU kernel.
- **Memory Throughput:** The fused implementation achieved a **9.9% higher** memory throughput.

We attribute this sharp increase in L1 hits to improved temporal locality. By enqueuing the GeLU kernel immediately after the bias addition and MatMul on the same stream, the output data residing in the GPU's L2 (and potentially L1) cache is "hot" when the GeLU kernel begins execution.

2. Execution Time

Despite the cache improvements, the impact on wall-clock execution time was marginal.

- **Cycles:** The fused GeLU required 4624 cycles compared to 4710 cycles for the baseline.
- **Duration:** This resulted in a time of $3.58\mu s$ versus $3.68\mu s$ for the baseline, a reduction of only **2.61%**.

Discussion and Trade-offs While the *Fused Add-LayerNorm* optimization achieved a $1.5\times$ speedup, the *Fused MatMul-GeLU* optimization yielded only a $\sim 2.6\%$ improvement. It is important to analyze why this distinction exists.

The Bottleneck of Global Memory Round-Trips: In the Add-LayerNorm fusion, we successfully eliminated a write to global memory. The residual result was kept in a register and immediately consumed by the normalization logic. In contrast, our MatMul-GeLU implementation could not eliminate the memory traffic. The `cublasSgemm` kernel writes the full $M \times N$ result matrix to Global Memory. The subsequent `fused_gelu_kernel` must then read that entire matrix back from Global Memory, apply the activation, and write it back.

Comparison to "True" Epilogue Fusion: To achieve significant speedups here, "Epilogue Fusion" is required. This is a technique supported by `CUTLASS` which we were able to leverage in other optimizations, such as Split-K. Because our implementation relied on the standard `cublasSgemm` and separate CUDA kernels, we paid the bandwidth penalty of the memory round-trip. The small gains we observed (2.6%) are purely due to the reduction of API overhead and the slight cache locality benefits of back-to-back kernel execution, rather than a fundamental reduction in memory traffic.

Conclusion Ultimately, this experiment demonstrates that minimizing kernel launch overhead via host-side wrappers yields diminishing returns for bandwidth-bound operations. While we successfully improved cache residency—evidenced by the 300% increase in L1 hit rates—the dominant cost remains the mandatory write to Global Memory by the cuBLAS kernel. To unlock the full potential of fusion for this sequence, future iterations would require transitioning to `CUTLASS` to utilize hardware-supported Epilogue Fusion, effectively bypassing Global Memory for intermediate results entirely.

7.1.3 Verifying Correctness of Kernel Fusions

Test Methodology To verify the correctness of our kernel fusions (*Fused Add-LayerNorm* and *Fused MatMul-GeLU*), we leveraged the existing verification infrastructure provided for the required optimizations. The general test structure remains identical to the baseline.

However, because these optimizations involve fusing multiple operations into single kernel launches, we modified the testing drivers to support these fused pathways:

- **Modified Driver:** We updated `gpt2.cuh` and `test_gpt2_kernels.cu`. Instead of invoking the individual kernel launchers sequentially (e.g., `residual_forward` → `layernorm_forward`), the driver now calls our fused wrappers (`fused_add_layernorm_forward` and `fused_matmul_gelu_forward`).

- **Memory Management:** The test harness was updated to handle the memory allocation changes to ensure each layer of the forward pass is connected properly.

TF32 Configuration It is important to note that while we enabled TensorFloat-32 (TF32) for our high-performance Flash Attention and MatMul kernels, we **disabled TF32** for the verification of these fused kernels. This ensures that the verification logic checks for strict FP32 numerical correctness without the precision loss inherent to TF32 operations.

Environment and SLURM Configuration The standard `verify.slurm` script can be used to run the functionality tests, and either the `test_gpt2` or the `test_gpt2_kernels` commands can be uncommented to test with those respective scripts. However, depending on the current state of the cluster environment (specifically regarding the recent Red Hat 9 upgrade), the following modifications to the SLURM script may be necessary to ensure the CUDA environment loads correctly:

1. **Update CUDA Version:** Change the module load command from `cuda/12.4` to `cuda/12.8`.
2. **Module Conflict Resolution:** Insert the command `module unload cudatoolkit` between lines 16 and 17 (immediately before loading the new CUDA module) to prevent conflicts with the legacy toolkit.

Listing 2: Modified SLURM Configuration

```
# ... (Lines 1-15)
module reset
# REQUIRED for RH9 Environment:
module unload cudatoolkit
module load cuda/12.8

# ... (Rest of script)
```

Running Verification The following is an ordered list of instructions which should make our kernel fusions easy to evaluate for correctness.

- Clone project repository to Delta
- Switch to branch `max-kernel-fusion`
- Perform `make` to compile
- Modify `verify.slurm` in the root directory as needed, such that it runs the desired test (`test_gpt2` or `test_gpt2_kernels`) and adheres to the changes described above if you are logged into a RedHat 9 node
- Dispatch job via `sbatch verify.slurm`
- View correctness results in `verify_output.out`

7.2 Proposed Optimization 2: CUTLASS Matrix Multiplication

7.2.1 Introduction and Motivation

CUTLASS is CUDA Templates for Linear Algebra Subroutines, which is NVIDIA's open source template. CUTLASS provides a highly customizable template. For example, we can modify the tile sizes, memory access patterns, operator classes, and epilogue operations. We investigated CUTLASS as a potential optimization for the matrix multiplication operation in GPT-2's forward pass. We think this will be useful for leveraging TF32 tensor cores for throughput and enabling kernel fusions.

7.2.2 Implementation Details

CUTLASS Configuration

```
//Headers
#include "cutlass/cutlass.h"                                  // core CUTLASS header
#include "cutlass/gemm/device/gemm.h"                  //The device::Gemm template class
#include "cutlass/epilogue/thread/linear_combination.h"//Post-GEMM Operation
```

```cpp
#include "cutlass/layout/matrix.h"                //Row-major, Column-major definitions
#include "cutlass/numeric_types.h"
#include "cutlass/arch/arch.h"
#include "cutlass/arch/mma.h"
#include "cutlass/util/device_memory.h"


// Type definitions (Make all the datatypes float)
using ElementA = float;
using ElementB = float;
using ElementC = float;
using ElementAccumulator = float;
using ElementCompute = float;

using LayoutA = cutlass::layout::RowMajor;     // Input is row-major
using LayoutB = cutlass::layout::ColumnMajor;  // Weight declared as col-major
using LayoutC = cutlass::layout::RowMajor;     // Output is row-major

//Used as a back-up option incase if tensor cores failed (since memory alignment issue before)
using CutlassGemmSIMT = cutlass::gemm::device::Gemm<
    ElementA, LayoutA,
    ElementB, LayoutB,
    ElementC, LayoutC,
    ElementAccumulator,
    cutlass::arch::OpClassSimt,              // CUDA cores (no alignment requirement)
    cutlass::arch::Sm80,                     //Architecture type depending on the GPU
    cutlass::gemm::GemmShape<128, 128, 8>,   // ThreadBlock shape
    cutlass::gemm::GemmShape<32, 64, 8>,     // Warp shape
    cutlass::gemm::GemmShape<1, 1, 1>,       // Instruction shape (scalar for SIMT)
    cutlass::epilogue::thread::LinearCombination<ElementC, 1, ElementAccumulator, ElementCompute>,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    2,   // Pipeline stages
    1,   // Alignment A = 1
    1    // Alignment B = 1
>;


using CutlassGemmTF32_Unaligned = cutlass::gemm::device::Gemm<
    ElementA, LayoutA,
    ElementB, LayoutB,
    ElementC, LayoutC,
    ElementAccumulator,
    cutlass::arch::OpClassTensorOp,
    cutlass::arch::Sm80,
    cutlass::gemm::GemmShape<128, 128, 32>,
    cutlass::gemm::GemmShape<64, 64, 32>,
    cutlass::gemm::GemmShape<16, 8, 8>,      // TF32 instruction
    cutlass::epilogue::thread::LinearCombination<ElementC, 1, ElementAccumulator, ElementCompute>,  // Epilogue width = 1
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    3,
    1,   // Alignment A = 1
    1    // Alignment B = 1
>;

//Element bias addition
__global__ void add_bias_kernel(float* out, const float* bias, int M, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < M * N) {
        out[idx] += bias[idx % N];
    }
}
```

```

//cublas as a back-up
inline void matmul_cublas(float* out, const float* inp, const float* weight,
                          int M, int N, int K) {
    const float alpha = 1.0f;
    const float beta = 0.0f;
    cublasSgemm(
        cublas_handle,
        CUBLAS_OP_T, CUBLAS_OP_N,
        N, M, K,
        &alpha,
        weight, K,
        inp, K,
        &beta,
        out, N
    );
}

//Preventing fails from showing up from cutlass launching, causing not able to use the back-up
inline void clear_cuda_error() {
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        // Error cleared, continue
    }
}


template<typename GemmKernel>
inline bool try_cutlass_gemm(float* out, const float* inp, const float* weight,
                             int M, int N, int K, bool verbose = false) {

    typename GemmKernel::Arguments args(  //Create for the cutlass arguments
        {M, N, K},
        {inp, K},
        {weight, K},
        {out, N},
        {out, N},
        {1.0f, 0.0f}
    );

    GemmKernel gemm;

    // Check if this configuration can handle the problem, used to debug the memory alignment issue
    cutlass::Status status = gemm.can_implement(args);
    if (status != cutlass::Status::kSuccess) {
        if (verbose) printf(" can_implement failed: %d\n", (int)status);
        return false;
    }

    // Allocate workspace
    size_t workspace_size = GemmKernel::get_workspace_size(args);
    cutlass::device_memory::allocation<uint8_t> workspace(workspace_size);

    // Initialize
    status = gemm.initialize(args, workspace.get());
    if (status != cutlass::Status::kSuccess) {
        if (verbose) printf("  initialize failed: %d\n", (int)status);
        return false;
    }

    // Execute
    status = gemm();
    if (status != cutlass::Status::kSuccess) {
```

```
        if (verbose) printf("  execute failed: %d\n", (int)status);
        clear_cuda_error();  // Clear any CUDA error from failed execution
        return false;
    }

    // Verifying whether launched successfully
    cudaError_t err = cudaDeviceSynchronize();
    if (err != cudaSuccess) {
        if (verbose) printf("  CUDA error: %s\n", cudaGetErrorString(err));
        clear_cuda_error();
        return false;
    }

    return true;
}

static bool first_call = true;
static int preferred_kernel = 0;  // 0=unknown, 1=TF32, 2=SIMT, 3=cuBLAS

void matmul_forward(float *out, const float *inp, const float *weight, const float *bias,
                    int B, int T, int C, int OC) {

    const int M = B * T;
    const int N = OC;
    const int K = C;

    bool success = false;

    // First call: determine which kernel works
    if (first_call) {
        first_call = false;

        // Check alignment (for alignment debugging)
        bool aligned = ((uintptr_t)inp % 16 == 0) &&
                       ((uintptr_t)weight % 16 == 0) &&
                       ((uintptr_t)out % 16 == 0);

        printf("\n[CUTLASS] First call: M=%d, N=%d, K=%d, aligned=%s\n",
               M, N, K, aligned ? "yes" : "NO");

        // Try TF32 with unaligned support first
        if (try_cutlass_gemm<CutlassGemmTF32_Unaligned>(out, inp, weight, M, N, K, true)) {
            printf("[CUTLASS] Using TF32 TensorOp (Alignment=1)\n\n");
            preferred_kernel = 1;
            success = true;
        }
        // Try SIMT
        else if (try_cutlass_gemm<CutlassGemmSIMT>(out, inp, weight, M, N, K, true)) {
            printf("[CUTLASS] Using SIMT (CUDA cores)\n\n");
            preferred_kernel = 2;
            success = true;
        }
        // Fall back to cuBLAS
        else {
            printf("[CUTLASS] All configs failed, using cuBLAS\n\n");
            preferred_kernel = 3;
            clear_cuda_error();  // Make sure errors are cleared before cuBLAS
        }
    } else {
        // Subsequent calls: use the kernel that worked
        switch (preferred_kernel) {
            case 1:
```

```
                success = try_cutlass_gemm<CutlassGemmTF32_Unaligned>(out, inp, weight, M, N, K);
                break;
            case 2:
                success = try_cutlass_gemm<CutlassGemmSIMT>(out, inp, weight, M, N, K);
                break;
            default:
                success = false;
                break;
        }
    }

    // Fallback to cuBLAS if CUTLASS failed
    if (!success) {
        clear_cuda_error();  // Clear any CUTLASS errors before calling cuBLAS
        matmul_cublas(out, inp, weight, M, N, K);
    }

    // Add bias
    if (bias != NULL) {
        int total = M * N;
        int block_size = 256;
        int grid = (total + block_size - 1) / block_size;
        add_bias_kernel<<<grid, block_size>>>(out, bias, M, N);
    }
}

#endif // __MATMUL_KERNEL_CUH__
}
```

7.2.3 Critical Challenge: Memory Alignment

During the functional test, we discovered that CUTLASS TF32 tensor core operations were failing with misaligned address errors (we are using 16 byte alignment initially).

```
    We inserted debug codes into it and found out there is a pointer alignment
    inp:    0x7f01ea0c0000  (%16=0) ALIGNED
    weight: 0x7f0205672000  (%16=0) ALIGNED
    out:    0x7f01f4c80418  (%16=8) NOT ALIGNED
```

The root cause is from GPT2's memory layout in gpt2.cuh. The test did contiguous allocation which contains some single float tensor allocation, and the test assign each tensor pointer as offset from base. This causing that the pointer will not be 16 byte alignment. Switching to 1 byte alignment will solve the issue but hurting the performance at the same time. The TF32 tensor cores load matrix using vectorized instructions, which requires 16-byte alignment. It can load 16 bytes in a single transaction. When we switch to 1-byte alignment, the transaction efficiency gets much lower.

7.2.4 Performance Analysis

Metric	cuBLAS Baseline	Optimized	Change
Duration (μs)	24.26	61.95	$+255.4\%$
Compute Throughput (%)	60.80	13.05	-78.54%
Memory Throughput (%)	61.28	26.98	-55.97%

Table 10: Impact of `CUTLASS` on matrix multiplication performance.

We can see there is a huge performance degradation on CUTLASS.

- The first cause of this huge degradation is the memory load. From the memory throughput we can found there is around 55% degradation, which is due to setting alignment equals to 1. Comparing to the alignment

of 16 bytes, the current alignment takes 4 times more memory instructions to load the same amount of data, which causing the low memory bandwidth usage.

- The second cause is that CUTLASS contains static tile configuration instead of runtime kernel selection. cuBLAS maintains hundreds of pre-compiled kernels with different sizes. It can select the optimal one at the runtime based on the exact matrix dimensions, memory alignment, available hardware resources. However, CUTLASS uses a single compile-time configuration, which showed in implementation section. GPT2 has varying matrix dimensions causing our configuration will perform worse when we encounter small matrix multiplication.

Overall, in GPT2 inference, we think that cuBLAS will perform better than CUTLASS because its runtime kernel selection adn good alignment handling. cuBLAS detects misalignment at runtime and will adjust to the appropriate code path without serious performance downgrade. Additionally, due to the limited time, we were not able to tune the configuration for better performance.

7.2.5 Potential Solution

Instread of modifying the gpt2.cuh file, to handle the alignment issue we can use hybrid approach which using CUTLASS and cuBLAS at the same time.

```
    bool aligned = ((uintptr_t)ptr % 16 == 0);
    if (aligned) use_cutlass();  // Alignment=4, vectorized
    else use_cublas();           // Always works
```

The main reason why we suggest to keep CUTLASS is because of the CUTLASS customization ability. It contains many additional functions in the template including kernel fusion, quantization, custom epilogues, and etc. It's high customize allowing users can implement more advanced features to improve the performance.

7.2.6 Verifying Correctness of CUTLASS

The CUTLASS branch passes the provided `test-gpt2-kernels` and `test-gpt2`. Below is the instruction for verification.

- Fetch the files in the branch cutlass (cutlass files already in the kernels folder)
- Make sure the makefile is the same as the branch one
- In terminal, type `make install-cutlass`
- Using command `make check-cutlass` to check if the cutlass is correctly installed.
- Build the target by typing `make all`
- Then you can run the verification by modifying `verify.slurm` and dispatch the job.
- View the correctness result in `verify_output.out`

7.3 Proposed Optimization 3: Activation Aware Quantization (AWQ)

7.3.1 Motivation

Activation-Aware Quantization (AWQ) is a post-training quantization technique designed specifically for large language models. The key observation is that activation outliers can cause large accuracy drops when weights are naïvely quantized. AWQ works by identifying which weight channels are most "salient" or sensitive to these activation spikes and selectively scaling or preserving precision for those channels while aggressively quantizing the rest. This makes quantization much more robust: the model maintains high accuracy even at low bit-widths (e.g., 4-bit), while achieving large reductions in memory footprint and faster inference. Because it requires no retraining and only a small calibration set, AWQ is practical, efficient, and especially useful for deploying LLMs on consumer GPUs or edge devices.

On the hardware side, quantizing weights to 8-bit significantly reduces DRAM traffic, which is the primary bottleneck in LLM inference. Lowering weight precision from FP32 to 8-bit directly decreases the amount of data fetched from off-chip memory during each matrix multiplication. This reduction in memory bandwidth demand leads to substantial throughput improvements, especially for memory-bound workloads such as autoregressive decoding.

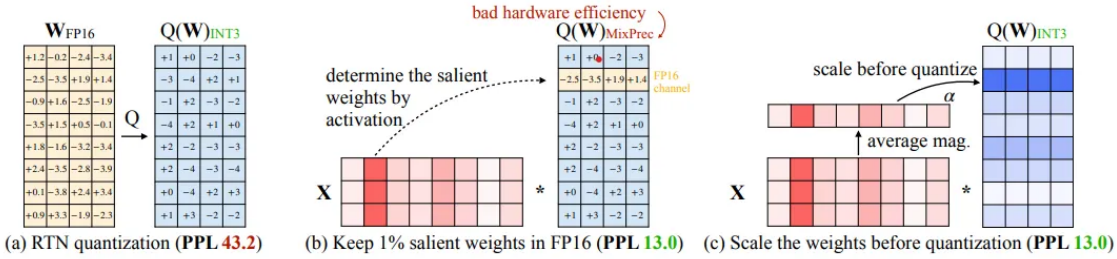

Figure 11: A visual diagram of AWQ from the original paper .

7.3.2 Implementation Details

AWQ was performed in the four following steps.

1. **Calibration**
2. **Optimal Scaling**
3. **Quantization**
4. **Inference-Time Dequantization**

Calibration Calibration was performed to capture the per-channel activation statistics required to identify salient weight channels. In particular, the LayerNorm (both pre-attention and pre-MLP), attention outputs, and GELU activations are monitored because they directly shape the magnitude and distribution of signals entering large linear projections, where activation outliers most strongly amplify quantization error. These activations therefore provide the most informative signal for identifying weight channels whose scaling is critical to preserving model accuracy under low-bit quantization. See `https://github.com/illinois-ece408/fa25_ece408_ClosedAI/blob/salient_awq/scripts_awq/awq_calibration.cu` for more details.

In alignment with the original AWQ paper, the Huggingface pile-val-backup dataset was used. Deviating from the paper, which used 512 samples with a maximum of 512 tokens each, 128 samples with a maximum token count of 256 were passed to the GPT2 model in batches of 32 due to resource constraints. The GPT2 model conducted 10 forward passes per sample. The absolute magnitude value per channel was accumulated and divided by the total number of tokens processed to produce the average magnitude per channel.

Optimal Scaling In AWQ, the authors performed a grid search over a scaling hyperparameter α that controls how strongly activation statistics influence per-channel weight scaling before quantization. For each layer, they collected the activation magnitudes, applied candidate α values to compute corresponding weight scaling factors, quantized the scaled weights, and then measured the mean squared error (MSE) between the original full-precision layer outputs and the quantized layer outputs. The α that minimizes this output MSE is selected, as it best balances suppressing activation outliers against preserving the original computation, yielding minimal accuracy degradation without retraining. Our process was the same, except the search space was reduced from 20 to 10 values due to resource constraints. See `https://github.com/illinois-ece408/fa25_ece408_ClosedAI/blob/salient_awq/scripts_awq/awq_grid_search.cu` for more details.

Quantization Once the optimal per-channel scales are determined, they are applied to the corresponding weight channels to adjust their dynamic range before quantization. Quantization parameters are then computed using asymmetric quantization, which, unlike symmetric quantization, includes a zero-point offset in addition to a scale factor to handle non-zero-centered ranges, for groups of channels of size 128 per the original AWQ paper. Finally, the weights are quantized to unsigned 8-bit integers but packed as unsigned 32-bit integers for efficiency. While the AWQ paper explores 4-bit quantization, this was deemed too extreme for our purposes. See `https://github.com/illinois-ece408/fa25_ece408_ClosedAI/blob/salient_awq/scripts_awq/awq_weight_quantization.cu` for more details.

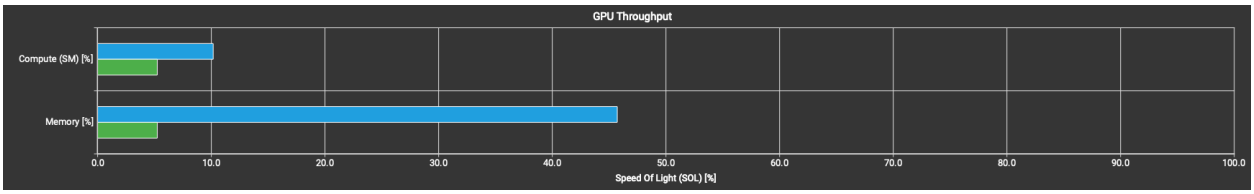

Figure 12:

Inference-Time Dequantization AWQ employs on-the-fly dequantization, converting quantized weights back to floating-point values dynamically during model inference. This approach allows us to store the quantized weights (along with their corresponding scale and zero-point parameters), retaining the memory and storage benefits of quantization while performing computations in floating-point precision. This helps preserve numerical accuracy, mitigates errors from low-bit quantization, and remains hardware efficient. See `https://github.com/illinois-ece408/fa25_ece408_ClosedAI/blob/salient_awq/kernels_awq/dequantize.cuh` for more detail.

7.3.3 Performance Analysis

As can be seen in the above figure, the matrix multiplication kernel shows a tremendous boost in throughput, especially regarding memory savings. While the overhead from dequantization led to more minor computational speed-ups, the overall performance boost is more than worth it.

7.3.4 Discussion and Trade-offs

Model quantization (AWQ or otherwise) offers major efficiency gains but comes with inherent tradeoffs. Reducing weights and activations to lower bit-widths dramatically cuts memory usage, bandwidth, and inference latency, enabling larger models to run on smaller hardware. However, quantization inevitably introduces numerical error, and while techniques like AWQ preserve accuracy by protecting sensitive channels, some degradation can still occur, especially for tasks requiring precise reasoning or long-context stability. Quantized models may also suffer from reduced robustness, harder fine-tuning, and increased complexity in deployment pipelines due to specialized kernels or calibration steps. In short, quantization improves efficiency but always balances speed and size against potential accuracy and flexibility loss.

```
============================================================
               AWQ MODEL COMPARISON TEST
============================================================

INPUT TEXT:
The meaning of life is

------------------------------------------------------------
STANDARD GPT2 MODEL OUTPUT:
------------------------------------------------------------
 that there is no end, and God does not know who he is.
 He knows us, but he does not know us, and he does not know us.
 God, therefore, is not able to teach him to know us, and God is

Metrics:
  - Perplexity: 7.173447
  - Generation time: 417.86 ms
  - Tokens/sec: 119.66

------------------------------------------------------------
AWQ QUANTIZED MODEL OUTPUT:
------------------------------------------------------------
Discussion"; standingclubDiscussionclubclub";
    clubclubbreakerclubemploymentclubkeeperDiscussionclub
conversationcemberclubclubclubclubclubclubclub guyclubclubDiscussionAGclub
    conversation";club courseclubclub";clubclubcemberDiscussionclubclub";clubument
    Dragonboundclub
```

```

Metrics:
  - Perplexity: 20.541591
  - Generation time: 271.54 ms
  - Tokens/sec: 184.13

============================================================
              MODEL PARAMETER SIZE COMPARISON
============================================================

Standard GPT2 (FP32):
  - Total parameters: 124,475,904
  - Model size: 474 MiB

AWQ Quantized Model:
  - Total parameters: 125,139,456
  - FP32 parameters: 40,204,800 (153 MiB)
  - INT8 parameters: 84,934,656 (81 MiB)
  - Total size: 234 MiB

Compression Summary:
  - Compression ratio: 2.03x
  - Size reduction: 50.6%
============================================================
```

As can be seen above, there's an almost 50% size reduction. However, this comes at a cost of higher perplexity.

7.3.5 Verifying Correctness

Perplexity is a standard metric used to evaluate how well a language model predicts a reference text. Given a sequence of tokens $\{x_1, x_2, \ldots, x_N\}$, a model produces a probability distribution $p_\theta(x_t \mid x_{<t})$ over the next token at each position. The quality of the model's predictions is measured by the *cross-entropy loss*:

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^{N} \log p_\theta(x_t \mid x_{<t}).$$

Perplexity is simply the exponential of this cross-entropy:

$$\mathrm{PPL} = \exp(\mathcal{L}).$$

Practically, to compute perplexity you feed the model each prefix $(x_1, \ldots, x_{t-1})$, obtain the logits, convert them to probabilities via softmax, extract the probability assigned to the correct token x_t, accumulate the log-probabilities across all positions, compute the average cross-entropy, and exponentiate. Lower perplexity indicates the model assigns higher probability to the correct tokens and therefore predicts the text better.

We applied this formula in awq_test.cuh in order to measure the difference in perplexity. To run the test, compile awq_test.cuh with the Makefile command 'make awq_test'. The script can be run in two modes. In the default, the AWQ model tries to complete the input text argument similar to next_generation_tokens.cu. When adding the "-c" argument before the input text, this does a side by side comparison with the default GPT-2 model on the same text for perplexity and speed.

7.3.6 Challenges

Managing dequantization overhead while minimizing accuracy loss is a central challenge in deploying AWQ-quantized models. On-the-fly dequantization introduces additional computation at inference, which can offset some of the latency gains from low-bit quantization, and improper scaling can propagate errors that degrade model performance. Balancing these factors requires careful tuning of quantization parameters and selective dequantization strategies to preserve accuracy. Future integration with libraries like CUTLASS could accelerate matrix operations on GPUs, reducing dequantization overhead and further improving the efficiency of low-bit inference.

REFERENCES

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.